

Supplementary Methods and Proofs

GL1F: Reproducible Integer Tree-Ensemble Inference on the EVM

Mikhail Fedorov

6 September 2026

Manuscript supplementary material; not peer reviewed

S1 Scope and notation

This supplement provides the byte-level definitions, supporting proofs, and experimental details for the accompanying article. A model is a complete, fixed-depth integer tree ensemble whose serialized core can be evaluated locally or reconstructed from EVM runtime-code chunks. The results concern representation and execution. They do not establish predictive validity of the application named in a model’s metadata.

The formal results distinguish the serialized byte string, the integer function it defines, and the registry record used to locate those bytes. Their equivalence is conditional on canonical parsing, exact reconstruction, consistent registry fields, identical packed inputs, and exact integer accumulation. They are handwritten proofs of evaluator specifications. Finite tests assess the implementations separately; neither the proofs nor the tests establish a verified compilation from source to deployed bytecode. Training uses floating-point arithmetic and is therefore covered by an explicit, finite conformance profile rather than a universal determinism claim.

S2 Binary layout

All multibyte integers are little-endian. Table S1 records the fixed header shared by the two supported core versions. The v1 header is followed immediately by scalar trees. In v2, the fixed header is followed by K signed int32 base values and then the trees, in output-major order.

Table S1: GL1F fixed header. Offsets and sizes are in bytes.

Offset	Size	Type	v1 scalar	v2 vector
0	4	Bytes	GL1F	GL1F
4	1	uint8	Version 1	Version 2
5	1	uint8	Reserved, zero	Reserved, zero
6	2	uint16	Feature count F	Feature count F
8	2	uint16	Depth d	Depth d
10	4	uint32	Tree count T	Trees per output R
14	4	int32	Base a_0	Reserved, zero
18	4	uint32	Scale s	Scale s
22	2	uint16	Reserved, zero	Output count K

For a complete tree of depth d , let $I(d) = 2^d - 1$ and $L(d) = 2^d$. The internal-node array contains $I(d)$ eight-byte records in breadth-first heap order. Within each record, the feature index occupies bytes 0–1, the signed int32 threshold occupies bytes 2–5, and bytes 6–7 are reserved and zero. The threshold is intentionally unaligned. The internal-node array is followed by $L(d)$ signed int32 leaf values. A tree therefore occupies $b(d) = 12 \cdot 2^d - 8$ bytes, with no inter-tree padding.

Using zero-based indices, tree t begins at $24 + tb(d)$ in v1. Tree t of output k begins at $24 + 4K + (kR + t)b(d)$ in v2. Relative to a tree’s first byte, internal node j begins at $8j$ and leaf u begins at $8I(d) + 4u$. These expressions permit bounds checking before decoding. The full grammar, including the distinction between wire-field limits and the narrower first-party deployment profile, is maintained in `FORMAT_SPEC.md`.

S3 Formal model and correctness results

This section separates three objects that are easy to conflate: (i) the mathematical forest, (ii) its GL1F byte representation, and (iii) a registry entry that points to EVM code chunks. The equivalence result for the first two is exact under the representation, input, depth, and arithmetic premises below. Its application to a registry entry is also conditional on a well-formed storage manifest, because the deployed registry does not itself recompute the model hash or validate every serialization invariant.

S3.1 Canonical representation

Let $\mathbb{I}_{32} = [-2^{31}, 2^{31} - 1] \cap \mathbb{Z}$. A complete binary tree of depth d contains

$$L(d) = 2^d \text{ leaves and } I(d) = 2^d - 1 \text{ internal nodes.} \quad (\text{S1})$$

An internal node occupies eight bytes: a little-endian unsigned 16-bit feature index, a little-endian signed 32-bit threshold, and two reserved bytes. A leaf occupies four bytes containing a little-endian signed 32-bit value. Consequently, one serialized tree occupies

$$b(d) = 8I(d) + 4L(d) = 12 \cdot 2^d - 8 \quad (\text{S2})$$

bytes.

Definition S3.1 (Canonical GL1F v1 model). *A canonical publication-profile scalar model is a byte string B consisting of a 24-byte header and T trees, where $F \geq 1$, $1 \leq d \leq 12$, and $0 \leq T \leq 2^{32} - 1$. A zero-tree v1 object is the well-defined base-only case. Its header contains magic **GL1F**, version one, feature count F , depth d , tree count T , base value $a_0 \in \mathbb{I}_{32}$, and scale $s \in [1, 2^{32} - 1] \cap \mathbb{Z}$. Reserved bytes are zero, every node feature is in $\{0, \dots, F - 1\}$, and*

$$|B| = 24 + Tb(d). \quad (\text{S3})$$

For a deployed-registry EVM application we additionally require equality between serialized and registered F, d, T, a_0 , and s , and $T \leq 2^{16} - 1$. The latter is a deployed-interface restriction: the v1 wire field for T is unsigned 32-bit, whereas the deployed registry stores total tree count as unsigned 16-bit.

Definition S3.2 (Canonical GL1F v2 model). *A canonical publication-profile vector model has $F \geq 1$, $1 \leq d \leq 12$, $K \geq 2$ outputs, $R \geq 1$ trees per output, base vector $(a_{0,0}, \dots, a_{0,K-1}) \in \mathbb{I}_{32}^K$, and output-major tree order. Its exact length is*

$$|B| = 24 + 4K + KRb(d). \quad (\text{S4})$$

*The header has magic **GL1F**, version two, positive F and s , and zero reserved fields. Every node feature is smaller than F . Registered feature count and scale equal the serialized values. Publication through the deployed registry additionally requires $KR \leq 2^{16} - 1$, because that interface stores the total tree count in a 16-bit field; this is narrower than the core format's separate 16-bit K and 32-bit R fields.*

The format does not contain a task discriminator. Identical v1 bytes can be described externally as regression or binary-logit inference, and identical v2 bytes can be described as multiclass or multilabel inference. Here keccak256 denotes Ethereum's legacy Keccak-256 function, not the standardized FIPS SHA3-256 variant. Under the usual collision- and second-preimage-resistance assumptions, keccak256(B) binds a practical identifier to the exact serialized bytes B . It is not a semantic normal form: distinct canonical cores can denote the same extensional integer function. The digest also does not bind a task label, feature semantics, preprocessing contract, dataset, or intended use. Those claims require a separately authenticated semantic manifest.

The explicit depth premise matches the current first-party interface. It is substantially narrower than the 16-bit wire field and prevents an apparently valid header from requesting an impractically large complete tree or an evaluator-specific shift result.

S3.2 Chunked EVM representation

Let c be the nominal payload of every nonfinal chunk and therefore the logical byte-offset stride; the final payload may be shorter. Let $N = \lceil |B|/c \rceil$. The canonical publisher deploys each payload through `ModelStore.write`, creating runtime code

$$C_j = \text{GL1C} \parallel B[jc : \min\{(j+1)c, |B|\}].$$

It then creates a table whose data are N 32-byte words; the low 20 bytes of word j are the address of C_j . The table is itself prefixed by **GL1C**.

Definition S3.3 (Well-formed storage manifest). *For an EVM state σ and registry record r in that state, a manifest (P, c, N, ℓ) is well formed for B at (σ, r) when:*

1. $P \neq 0$, $4 \leq c \leq 24572$, $1 \leq N \leq 767$, $\ell = |B|$, and $N = \lceil \ell/c \rceil$;
2. table address P has runtime code in σ of exact length $4 + 32N$, beginning with **GL1C**;
3. its j th word has zero high 12 bytes and resolves through its low 20 bytes to nonzero address C_j ;
4. every nonfinal C_j in σ has exact code $\text{GL1C} \| B[jc : (j+1)c]$, and the final chunk has the exact code $\text{GL1C} \| B[(N-1)c : \ell]$; and
5. the corresponding fields of r equal P, c, N, ℓ .

The lower bound $c \geq 4$ is semantic, not cosmetic. Runtime primitive fields are at most four bytes, while the implemented boundary reader consults at most two chunks. A field could span more than two logical chunks if $c < 4$.

Lemma S3.4 (Pointer capacity). *A pointer table created by the deployed storage contract contains at most*

$$\left\lfloor \frac{24572}{32} \right\rfloor = 767$$

chunk addresses.

Proof. **ModelStore.write** accepts at most 24572 data bytes, and each table slot occupies exactly 32 data bytes. The four-byte **GL1C** prefix is added by the store and is not part of the payload limit. $\square$

For a manifest satisfying Definition S3.3, define the specified boundary reader $\text{Read}_{\sigma, M}(o, n)$ to compute $j = \lfloor o/c \rfloor$ and $r_o = o \bmod c$, copy from code offset $4 + r_o$ in chunk j , and, only when $r_o + n > c$, concatenate the remaining bytes from code offset four in chunk $j + 1$.

Lemma S3.5 (Exact byte reconstruction). *Given Definition S3.3, every runtime primitive read of $n \in \{1, 2, 4\}$ bytes at a model offset $o \in \mathbb{Z}_{\geq 0}$ satisfying $o + n \leq |B|$ returns exactly $B[o : o + n]$.*

Proof. By Euclidean division, write $o = jc + r$ with $j = \lfloor o/c \rfloor$ and $0 \leq r < c$. If $r + n \leq c$, the runtime resolves slot j and copies from code offset $4 + r$, which is exactly $B[o : o + n]$. Otherwise it copies $c - r$ bytes from chunk j and $n - (c - r)$ bytes from the start of chunk $j + 1$. Since $n \leq 4 \leq c$, no third chunk is required. The exact chunk-length and ordering clauses in Definition S3.3 make the concatenation equal to $B[o : o + n]$. $\square$

Remark S3.6. The **GL1C** check detects zero and many accidental pointers, but it is not content authentication. Arbitrary EVM code beginning with the same four bytes passes that test. Exact reconstruction is therefore a property of the manifest conditions, not of the magic prefix alone.

S3.3 Traversal and termination

Let i_j be the heap index after j decisions, with $i_0 = 0$. For packed feature x_q and threshold θ_q , a node updates

$$i_{j+1} = \begin{cases} 2i_j + 2, & x_q > \theta_q, \\ 2i_j + 1, & x_q \leq \theta_q. \end{cases} \quad (\text{S5})$$

Lemma S3.7 (Traversal interval). *After j decisions,*

$$2^j - 1 \leq i_j \leq 2^{j+1} - 2. \quad (\text{S6})$$

Proof. For $j = 0$, both bounds equal zero. Assume the claim at j . The smallest possible child is $2(2^j - 1) + 1 = 2^{j+1} - 1$, and the largest is $2(2^{j+1} - 2) + 2 = 2^{j+2} - 2$. These are precisely the bounds for $j + 1$. $\square$

Corollary S3.8 (Leaf-index safety). *After exactly d decisions,*

$$0 \leq i_d - I(d) \leq L(d) - 1.$$

Thus a traversal of a well-formed tree selects exactly one serialized leaf and cannot address outside that tree’s leaf array.

Proof. Substitute $j = d$ in Lemma S3.7 and subtract $I(d) = 2^d - 1$. $\square$

Lemma S3.9 (Forced-subtree inertness). *Suppose a terminal node denoting signed int32 value v is replaced by a complete forced continuation whose remaining internal nodes use feature zero and threshold $2^{31} - 1$, and whose descendant leaves all contain that same v . For every $q \in \mathbb{I}_{32}^F$, traversal through that continuation returns v ; hence the replacement preserves the terminal value over the entire packed-input domain.*

Proof. Definitions S3.1 and S3.2 give $F \geq 1$, so feature zero exists. Because $q_0 \leq 2^{31} - 1$, the strict predicate $q_0 > 2^{31} - 1$ is false at every inserted node and traversal follows the left child at each remaining level. The reached descendant leaf contains v . Moreover, assigning v to every descendant leaf makes the value independent even of the particular descendant selected. Thus the forced continuation cannot change the tree output for any int32 input vector. $\square$

Theorem S3.10 (Finite work and logical traversal state). *For a canonical v1 model, inference performs exactly Td node decisions and T leaf reads. For a canonical v2 model, inference performs exactly KRd node decisions and KR leaf reads. An abstract streaming evaluator that does not buffer the model needs $O(1)$ scalar logical traversal state and $O(K)$ vector logical traversal state, excluding input storage and implementation-specific read buffers. Model space is $24 + Tb(d)$ for v1 and $24 + 4K + KRb(d)$ for v2.*

Proof. Each complete tree executes the loop in Equation S5 exactly d times, followed by one leaf read. The scalar outer loop visits T trees. The vector loops visit R trees for each of K outputs. No inference loop is recursive or conditioned on a model value other than the bounded header counts. A streaming evaluator stores one scalar accumulator or a length- K result vector together with a constant number of indices and decoded primitive values. The model-space formulae follow from Equation S2. $\square$

Remark S3.11 (Bounded does not imply affordable). The theorem establishes a finite operation count. It does not guarantee that an arbitrary encoded dimension fits a transaction gas limit. The publication profile and empirical gas envelope must be reported separately. The deployed Solidity boundary-reader allocates a fresh dynamic byte buffer when one primitive spans two chunks. Since EVM memory allocation is monotonic within a call, its transient memory may grow linearly with the number of such boundary reads even though every primitive read is at most 32 bytes. This is an implementation cost, not additional logical traversal state.

S3.4 Cross-runtime equivalence

A *conforming forest evaluator* uses the specified little-endian signed/unsigned decoding and strict comparison, accumulates the mathematical integer sums below without overflow or loss of precision, and selects the lowest index among equal maximum logits when returning a class. A conforming local evaluator first validates the version-specific canonical definition. A *conforming EVM evaluator* at (σ, r) obtains every core primitive through $\text{Read}_{\sigma, M}$, uses the version-aware registry relations below, and otherwise applies the same forest semantics. These are specification definitions; source inspection and finite compiled/deployed tests separately assess the deployed **ForestRuntime** path. Client-side preprocessing may construct the packed integer vector, but real-number quantization is not part of the theorem. Applying the theorem to an external endpoint additionally requires its enabled, authorization, and fee conditions to reach the internal evaluator with the packed input unchanged.

For a scalar model, define the integer function

$$f_B(q) = a_0 + \sum_{t=0}^{T-1} v_{t, \pi_t(q)}, \quad (\text{S7})$$

where $\pi_t(q)$ is the leaf selected by Equation S5. For a vector model,

$$f_{B,k}(q) = a_{0,k} + \sum_{t=0}^{R-1} v_{k,t,\pi_{k,t}(q)}. \quad (\text{S8})$$

Theorem S3.12 (Specification-level conforming-evaluator equivalence). *Let B be canonical under Definition S3.1 or Definition S3.2, let the EVM manifest satisfy Definition S3.3 at the same (σ, r) used for evaluation, and require the following nine version-aware relations for r*

$$\begin{aligned} nFeatures &= F, & nTrees &= \begin{cases} T & (v1), \\ KR & (v2), \end{cases} & depth &= d, \\ baseQ &= \begin{cases} a_0 & (v1), \\ 0 & (v2 \text{ registry convention}), \end{cases} & scaleQ &= s, & tablePtr &= P, \\ totalBytes &= \ell, & numChunks &= N, & chunkSize &= c. \end{aligned}$$

Let every evaluator receive the identical $4F$ -byte sequence formed by packing $q \in \mathbb{I}_{32}^F$ as signed little-endian $int32$ values. Then conforming local and EVM evaluators return $f_B(q)$ for $v1$ and $(f_{B,k}(q))_{k=0}^{K-1}$ for $v2$. Their $v2$ class operation returns

$$\left(\min \arg \max_{0 \leq k < K} f_{B,k}(q), \max_{0 \leq k < K} f_{B,k}(q) \right),$$

where the minimum implements the specified lowest-index tie rule.

Proof. Lemma S3.5 gives identical serialized bytes. The $v2$ EVM and local evaluators decode the same header, node, threshold, and leaf integers. The conforming $v1$ EVM evaluator uses registered inference fields rather than parsing the header; the theorem’s nine-relation premise makes those fields identical to the local decoder’s values, while the node, threshold, and leaf integers still follow from Lemma S3.5. At the root, every evaluator compares the same q_f and θ_q using the same strict predicate. If their indices agree before a level, the identical predicate and Equation S5 give the same next index. Induction and Corollary S3.8 give the same selected leaf for every tree. Finally, Equations S7 and S8 add the same finite integer sequence to the same base term, with the exact-accumulation premise preserving that mathematical sum. Applying the same lowest-index argmax scan to the identical $v2$ vector gives the same class index and maximum integer logit. $\square$

Remark S3.13. This handwritten theorem concerns conforming evaluator specifications in one state and record. Source inspection supports the mapping from the deployed Solidity path to that specification; compiled tests and pinned observations supply finite implementation evidence. None is a mechanized EVM-bytecode proof, verified Solidity-to-bytecode refinement, or source-to-live-bytecode provenance proof.

Proposition S3.14 (Exact JavaScript accumulation under the deployment profile). *All integer accumulators permitted by the canonical ModelStore/registry profile are strictly within JavaScript’s exact integer range.*

Proof. For $v1$, the registry’s 16-bit tree count gives

$$|f_B(q)| \leq (65535 + 1)2^{31} = 2^{47} < 2^{53}.$$

For $v2$ with $d \geq 1$, Equation S2 gives $b(d) \geq 16$. Lemma S3.4 bounds total core payload by $767 \cdot 24572 = 18846724$ bytes. Since $K \geq 2$,

$$R < \frac{18846724}{2 \cdot 16} < 589000.$$

Hence every output and every partial sum obeys

$$|f_{B,k}(q)| < (589001)2^{31} < 2^{51} < 2^{53}.$$

All $int32$ inputs and summands are themselves exactly representable. $\square$

Remark S3.15. The theorem concerns inference from fixed bytes. Byte-identical training is a separate empirical claim because split scoring, gradients, and early stopping use floating-point arithmetic. It requires fixed preprocessing, operation order, feature/class order, seeded random-number consumption, and compiler flags, plus a parity test matrix.

S3.5 Fixed-point approximation

For finite $z \in \mathbb{R}$ and integer $s > 0$, define the abstract exact-real quantizer with JavaScript’s half-toward-positive-infinity tie rule

$$Q_s(z) = \text{clamp}_{\mathbb{I}_{32}} \left(\left\lfloor sz + \frac{1}{2} \right\rfloor \right). \quad (\text{S9})$$

This definition includes the asymmetric negative-half convention of `Math.round`; for example, $Q_1(-1.5) = -1$.

This is a mathematical abstraction. Production JavaScript first computes the binary64 product $\text{fl}(sz) = sz + \delta_s(z)$ and then applies `Math.round`. The two integers agree when that multiplication perturbation does not cross an integer rounding boundary. Without that premise, and provided the rounded production result remains in the int32 range, the direct implementation bound is

$$\left| \frac{\text{Math.round}(\text{fl}(sz))}{s} - z \right| \leq \frac{1/2 + |\delta_s(z)|}{s},$$

which can be slightly larger than $1/(2s)$. If int32 saturation occurs, neither bound applies.

Lemma S3.16 (Abstract single-value quantization error). *If Equation S9 does not saturate, then*

$$\left| \frac{Q_s(z)}{s} - z \right| \leq \frac{1}{2s}.$$

Proof. For $u = sz$, the integer $\lfloor u + 1/2 \rfloor$ lies in $[u - 1/2, u + 1/2]$. Divide by positive s . □

Theorem S3.17 (Stable-path output error). *Consider an ideal scalar forest whose base and the T leaves selected for input x are independently quantized by Equation S9 without saturation at common scale s . If quantization does not change any selected path, the dequantized GL1F output $\hat{y}$ satisfies*

$$|\hat{y} - y| \leq \frac{T + 1}{2s}.$$

For one output of a v2 model, replace T by R .

Proof. Apply Lemma S3.16 to the base and every selected leaf, then use the triangle inequality. □

Lemma S3.18 (Sufficient split-margin condition). *Suppose neither x nor threshold θ saturates. If*

$$|x - \theta| > \frac{1}{s},$$

then the truth value of $x > \theta$ equals that of $Q_s(x) > Q_s(\theta)$.

Proof. If $x > \theta + 1/s$, Lemma S3.16 implies $Q_s(x)/s \geq x - 1/(2s) > \theta + 1/(2s) \geq Q_s(\theta)/s$. The reverse ordering is analogous. □

Corollary S3.19 (Argmax stability). *Assume stable paths and unsaturated base/leaf quantization for a v2 model. If the difference between the largest and second-largest ideal logits exceeds*

$$\frac{R + 1}{s},$$

the GL1F integer argmax returns the same class. Equal integer logits are resolved deterministically in favor of the lowest class index.

Proof. Theorem S3.17 perturbs each of the two logits by at most $(R + 1)/(2s)$. Their order therefore cannot reverse under the stated margin. The implementation replaces the incumbent only on strict improvement, which establishes the tie rule. □

Remark S3.20 (Why a global error bound needs a margin). Near a split, quantization can select a different leaf. The difference between two leaves is not bounded by the rounding error alone. Therefore Theorem S3.17 must not be quoted without its stable-path condition unless an additional bound on alternate-leaf values is supplied. For production binary64 preprocessing, replace every $1/(2s)$ term above by $(1/2 + |\delta_s(z)|)/s$ for the corresponding quantized value, unless the implementation is shown to emit the same integer as the abstract quantizer; either alternative still requires that the corresponding int32 clamp not saturate. Provided neither operand saturates, a sufficient production split margin is therefore

$$|x - \theta| > \frac{1 + |\delta_s(x)| + |\delta_s(\theta)|}{s}.$$

S3.6 Limits of what the current registry proves

For clarity, the live verifier names nine relations rather than using an undefined “applicable metadata” category. It checks

$$\begin{aligned} nFeatures &= F, & nTrees &= \begin{cases} T & (v1), \\ KR & (v2), \end{cases} & depth &= d, \\ baseQ &= \begin{cases} a_0 & (v1), \\ 0 & (v2 \text{ reserved-field convention}), \end{cases} & scaleQ &= s, \\ tablePtr &= P, & totalBytes &= \ell, & numChunks &= N, & chunkSize &= c. \end{aligned}$$

The first five reconcile runtime registry metadata with the decoded core; the last four reconcile the runtime and byte-information registry getters with the storage manifest.

Proposition S3.21 (No unconditional content binding). *For the deployed registry entry point, the existence of a record under identifier h does not imply $h = \text{keccak256}(B)$ for the bytes reconstructed from its pointer table.*

Proof. The registration predicate checks only that $h \neq 0$ and that no prior record exists under h ; it accepts h and the pointer metadata as independent arguments. Choose any nonzero h and any accepted pointer tuple reconstructing B with $h \neq \text{keccak256}(B)$. The call can satisfy the remaining fee, metadata, ToS, and licence checks. Thus a valid transaction is a counterexample to the implication. $\square$

Remark S3.22 (Executable assurance-boundary witness). The isolated-chain integration test makes the counterexample concrete. The unmodified registry accepted this identifier:

```
0x60c713e19362bcb16b04fb6fc611821a
8a3fb90e1698490bf4819c2c44838875
```

for a pointer tuple whose 104 reconstructed bytes have hash

```
0xeb0ee33a46399a8592b6676035a7e580
c1f90e7761a2eb1f4a56d0f9f0e4ab18.
```

The runtime followed that record and returned integer prediction 1043. The production JavaScript chain loader now recomputes $\text{keccak256}(B)$ after exact reconstruction and rejects this mismatch before returning model bytes. This client-side check mitigates the deployed assurance gap; it does not change the registry predicate.

Corollary S3.23 (Need for independent verification). *Theorem S3.12 applies to a registry entry only after an independent verifier establishes Definitions S3.1 or S3.2, Definition S3.3, and all nine relations above. If the registry key is additionally claimed as a content identifier, that separate deployment-identity claim requires $h = \text{keccak256}(B)$; this equality is not a premise of execution equivalence for the reconstructed bytes.*

Proposition S3.24 (Open model storage permits independent local inference). *Under the documented GL1C storage design, an observer with chain-read access can reconstruct the intentionally public model and evaluate it locally, independently of the canonical runtime endpoint.*

Proof. The pointer table and every chunk are public EVM runtime code retrievable by address. Definition S3.3 gives a constructive algorithm for reassembling B . Equations S7 and S8 then define a local evaluator using only public integers. No secret required by evaluation is held by the fee-gated contract. $\square$

Remark S3.25. Mode-2 payment can still meter use of an official transaction endpoint, produce an on-chain event, or support service-level economics. It is not model encryption, copy protection, or exclusive inference.

S3.7 Verification requirements

Applying the equivalence theorem to a concrete deployment requires an exact core and storage reconstruction, canonical parser validation, agreement of all nine registry/core/storage relations, and identical packed inputs. A separately claimed content identifier requires recomputing the core’s Keccak-256 digest. Core bytes, chunk code, ordered addresses, registry responses, packed vectors, and expected integer outputs allow these conditions to be checked independently from a retained observation.

Source revision and compiler settings identify the build used in local tests. Creation and deployed bytecode, together with their hashes, are needed to examine any stronger source-to-deployment claim. Byte-identical training results remain empirical evidence separate from the inference theorem. These distinctions allow a failed reproduction to be assigned to a specific representation, state, input, arithmetic, or implementation premise.

S4 Training conformance and numerical corrections

The publication suite invokes the production browser worker under Node.js, compiles the production C++ trainer, invokes the Python command-line trainer, and compares the serialized core bytes. Its archived matrix contains 25 distinct three-engine training profiles and five auxiliary controls, giving 30 profile executions, together with one standalone IEEE-754 operation-order witness. The 18 test methods cover regression, binary classification, multiclass and multilabel objectives; linear and quantile binning; weighting; stratification; early stopping; fixed-budget refitting; learning-rate schedules; seed zero; repeatability; v1/v2 inference; rounding boundaries; packaging; and malformed-stream rejection. Profiles and test methods are different counting units.

The tested Linux environment used Python 3.12.13, NumPy 2.3.5, Node.js 24.19.0, and g++ 13.3.0 with C++17, `-ffp-contract=off`, and `-fno-fast-math`. Equality depends on the stated preprocessing, operation order, feature and class order, random-number consumption, and compiler settings. No evidence here establishes equality for every input, platform, compiler, or mathematical library.

S4.1 Histogram arithmetic

A finite arithmetic witness distinguished multiplying by a cached reciprocal from dividing by the range before multiplication in linear histogram binning. These expressions are equivalent over the reals but need not produce the same binary64 result at a bin boundary. The browser implementation was changed to use the same divide-then-multiply order as Python and C++. The witness is retained separately from the training-profile count.

S4.2 Multiclass probability computation

A generated 4,000-row, 12-feature, four-class profile exposed a second operation-order discrepancy. Python accumulated the softmax denominator from binary32-rounded exponentials, whereas JavaScript and C++ accumulated binary64 exponentials. One byte of the 22,120-byte Python core differed, representing a one-unit change in a quantized leaf. The corrected profile uses binary64 exponentials, an ascending-class-index binary64 denominator sum, a binary32 numerator, binary64 division, and a binary32 result. All three engines emit a core with SHA-256 digest

58e3d9e63c5dd1313872e0c5a07c229b071272f1137f393a0384601329a432eb

for that fixture. This is a regression result for a specified computation.

S4.3 Rounding and input boundaries

For a finite binary64 argument z , the Python and C++ rounding helpers match JavaScript’s `Math.round` integer result before int32 saturation, including its half-toward-positive-infinity convention. Directly evaluating `floor(z + 0.5)` in binary64 introduces an additional rounding step: for $z = 0.49999999999999994$, that expression returns one whereas `Math.round(z)` returns zero. The corrected helpers compare $z - \lfloor z \rfloor$ with 0.5 and enforce the relevant integer bounds. The exact-real definition in Equation S9 remains a mathematical abstraction; the preceding product-rounding correction still applies to production preprocessing.

The browser form preserves seed zero, and the strict Python GL1X loader rejects nonstandard JSON constants such as `NaN` and `Infinity`. These corrections align the tested input boundaries; they do not extend the determinism claim beyond the documented finite-input profile. The inference specification and deployed contracts were unchanged.

S5 Experimental methods and additional results

S5.1 End-to-end trainer timing

Each timing run used 3,000 rows, 12 features, and 60 depth-4 trees. For each engine and task, one warm-up preceded 30 measured repetitions. The measured interval included process startup, parsing, training, serialization, and output writing. Python and C++ read the same CSV input; JavaScript read an equivalent JSON numeric matrix, so these timings are not a pure training-kernel comparison. Engine order was rotated between repetitions. The trainer code was single-threaded, and host scheduling was not pinned. The host reported an AMD EPYC 9V74 processor with nine logical CPUs visible to the execution environment; the measurements do not characterize the performance of the entire physical server.

Table S2: End-to-end trainer median (interquartile-range width), in seconds.

Workload	Python	C++	JavaScript/Node.js
Regression	0.3046 (0.0124)	0.0365 (0.0017)	0.1384 (0.0088)
Binary classification	0.3149 (0.0177)	0.0437 (0.0027)	0.1610 (0.0130)

All three engines produced byte-identical 11,064-byte cores within each workload. Ratios of the observed Python and C++ medians were 8.35 for regression and 7.20 for binary classification. These are ratios of end-to-end times on two small workloads, not portable training speedups. The individual durations, parameters, model digests, and environment are stored in `benchmarks/results/publication_benchmark.json`. The recorded source revision is 72e91a5ba6d335fa5c8cc805c2ac40eb94bbc0fb, with a clean working tree.

S5.2 Isolated EVM integration

The integration test compiles the unmodified contracts using Solidity 0.8.20, optimizer 200, `viaIR=true`, and the Istanbul EVM target. It deploys ModelStore, ModelRegistry, ModelNFT, and ForestRuntime to an isolated Ganache 7.9.2 chain. Minimal canonical v1 and v2 cores are stored in 17-byte GL1C chunks and pointer tables, reconstructed, and checked against their content identifiers. An independent bigint reference evaluator is compared with scalar, vector, and class contract results on six vectors covering threshold equality and integer extremes. These are local test transactions and read calls.

Table S3: Isolated EVM correctness-fixture measurements.

Measurement	v1 scalar	v2 vector/class
Core bytes / chunks	104 / 7	156 / 10
Mean view gas estimate	77,600	109,011 vector; 108,412 class
Transaction inference gas	79,573	115,527

The 18 view comparisons matched. Equal vector logits select class zero, consistent with the lowest-index tie rule. The same test constructs the registry counterexample described in Proposition S3.21. Complete local results and source-input digests are stored in `benchmarks/results/evm_integration.json`.

S5.3 Scalar inference scaling

A complementary local benchmark publishes six deterministic two-feature v1 shapes using 20,000-byte chunks and evaluates the same 12 seeded vectors for each shape. All 72 reference/runtime comparisons matched. Gas estimates refer to the same recorded Solidity/Ganache profile as above.

For the tested scalar decoder, model-code reads within the tree bodies comprise one feature read and one threshold read per visited node, plus one leaf read per tree. Their count is $R_{\text{tree}} = T(2d + 1)$,

Table S4: Local scalar scaling observations. Mean gas estimates are rounded to the nearest gas unit.

Trees	Depth	Core bytes	Tree-body reads	Mean gas
20	3	1,784	140	286,247
50	3	4,424	350	644,454
50	4	9,224	450	810,777
100	4	18,424	900	1,576,945
200	4	36,824	1,800	3,121,210
50	6	38,024	650	1,146,414

excluding registry, header, and pointer-resolution accesses. A descriptive ordinary-least-squares fit to the six unrounded means was

$$\widehat{\text{gas}} \approx 42,900 + 1,708.3R_{\text{tree}},$$

with $R^2 \approx 0.99998$ and maximum absolute relative residual 1.45%. This six-point fit describes the tested shapes; it is not a gas bound or a forecast of live-network fees. Raw observations and unrounded statistics are stored in `benchmarks/results/evm_scaling_benchmark.json`.

S6 Historical deployment observation

The repository retains factual summaries of a provider-attested observation at GenesisL1 block 13,342,043, with recorded hash `0xffd825db1bb2534052a604db9584d361111d8bc9e19d753b0ee3861bf320d1b9`, in `benchmarks/results/LIVE_CHAIN_WITNESS.md` and `benchmarks/results/LIVE_CHAIN_WITNESS_EXTENDED_V2.md`. Those summaries report reconstruction of 31,185,324 bytes across 12 active models and matching local/provider-returned integer outputs. The raw provider responses and reconstructed bytes for that historical observation were not retained in the repository. Consequently, it cannot be replayed offline and is not used as the principal reproducibility evidence for this article. The model names in those records identify registered artifacts; they do not establish the accuracy of the named prediction tasks.

S7 Reproduction

The repository’s `REPRODUCIBILITY.md` specifies the environment, installation procedure, individual evidence commands, and archived result files. After installing those dependencies, the following commands run the local verification suite and regenerate the timing and scalar-scaling records:

```
make verify
python3 benchmarks/publication_benchmark.py \
  --rows 3000 --features 12 --trees 60 --repeats 30 \
  --out benchmarks/results/publication_benchmark.json
npm run benchmark:evm
```

These commands use local processes and an isolated EVM. They do not submit transactions to GenesisL1. The checked-in parity and local integration records include source-input SHA-256 digests; the verification suite checks those digests and the numerical invariants used in the article. A fresh network observation requires separate read-only RPC access and must be identified by its own block and retained responses. Provider-returned state is distinct from an independently verified proof of chain consensus or finality.